

Creditcardfraud_samefeatures

June 30, 2026

```
[1]: import pandas as pd
import numpy as np

# 1. Load both datasets
df_train_raw = pd.read_csv('/Users/mac/Desktop/IITK/Academics/SUMMER SEM_
↳2025-26/EC0723/GROUP PROJECT/archive/fraudTrain.csv')
df_test_raw = pd.read_csv('/Users/mac/Desktop/IITK/Academics/SUMMER SEM 2025-26/
↳EC0723/GROUP PROJECT/archive/fraudTest.csv')

# 2. Tag the data so you can easily separate it later
df_train_raw['is_test_set'] = 0
df_test_raw['is_test_set'] = 1

# 3. Concatenate into one massive dataframe for feature engineering
df = pd.concat([df_train_raw, df_test_raw], ignore_index=True)

# 4. Convert time and sort strictly by card number and time
df['trans_date_trans_time'] = pd.to_datetime(df['unix_time'], unit='s')
df = df.sort_values(by=['cc_num', 'trans_date_trans_time']).
↳reset_index(drop=True)

print(f"Combined Dataset Shape: {df.shape}")
```

Combined Dataset Shape: (1852394, 24)

example 1 (sort by last 5 transactions)

```
[2]: # Rolling mean of the 'amt' column over a 5-transaction window
df['HOBA_amt_mean_5tx'] = df.groupby('cc_num')['amt'].transform(
    lambda x: x.rolling(window=5, min_periods=1).mean()
)

# You would repeat this pattern for max, sum, and standard deviation.
```

example 2 (sort by past 1 day transactions)

```
[3]: # Set the index temporarily to execute the time-based rolling window
df_time_indexed = df.set_index('trans_date_trans_time')
```

```
# Calculate the rolling sum over '1D' (1 Day)
df['HOBA_amt_sum_1d'] = df_time_indexed.groupby('cc_num')['amt'].transform(
    lambda x: x.rolling('1D').sum()
).values
```

```
[4]: # Rolling max amount for the past 7 days, isolated by BOTH cardholder and
      ↪ transaction category
df['HOBA_amt_max_7d_by_category'] = df_time_indexed.groupby(['cc_num',
      ↪ 'category'])['amt'].transform(
    lambda x: x.rolling('7D').max()
).values
```

```
[5]: df_time_indexed = df.set_index('trans_date_trans_time')
```

```
[6]: def haversine_vectorized(lat1, lon1, lat2, lon2):
      # Convert decimal degrees to radians
      lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])

      # Haversine formula
      dlat = lat2 - lat1
      dlon = lon2 - lon1
      a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
      c = 2 * np.arcsin(np.sqrt(a))
      r = 6371 # Radius of earth in kilometers

      return c * r

      # Apply the function to create a base distance metric for the current
      ↪ transaction
df['distance_to_merchant_km'] = haversine_vectorized(
    df['lat'], df['long'],
    df['merch_lat'], df['merch_long']
)
```

```
[7]: df_time_indexed = df.set_index('trans_date_trans_time').sort_index()
```

```
[8]: # Assuming df_time_indexed is your dataframe with trans_datetime as the index
      # Calculate the maximum distance to a merchant over the past 3 days for each
      ↪ cardholder
df['HOBA_dist_max_3d'] = df_time_indexed.
    ↪ groupby('cc_num')['distance_to_merchant_km'].transform(lambda x: x.
    ↪ rolling('3D').max()).values

      # Calculate the average distance to merchants over the past 5 transactions
df['HOBA_dist_mean_5tx'] = df.groupby('cc_num')['distance_to_merchant_km'].
    ↪ transform(lambda x: x.rolling(window=5, min_periods=1).mean())
```

```
[9]: from sklearn.preprocessing import MinMaxScaler

# 1. Split the data back into train and test using the tag
df_train = df[df['is_test_set'] == 0].copy()
df_test = df[df['is_test_set'] == 1].copy()

# 2. Define columns that the model MUST NOT see (including our temporary tag)
columns_to_drop = [
    'is_fraud', 'is_test_set', 'cc_num', 'trans_num', 'merchant', 'category',
    'first', 'last', 'gender', 'street', 'city', 'state', 'job', 'dob',
    'trans_date_trans_time', 'unix_time', 'lat', 'long', 'merch_lat',
    ↪ 'merch_long',
    'Unnamed: 0'
]

# 3. Dynamically extract all numerical feature columns
feature_columns = sorted(
    [col for col in df_train.columns if col not in columns_to_drop]
)
FINAL_HOBA_FEATURES = feature_columns.copy()
print("=" * 60)
print(f"Final HOBA Feature Count = {len(feature_columns)}")
print("=" * 60)

# 4. Initialize and fit the scaler ONLY on the training data
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(df_train[feature_columns])

# 5. Transform the test data
X_test_scaled = scaler.transform(df_test[feature_columns])

# check if training and testing dataset are using different datasets
assert list(df_train[feature_columns].columns) == list(df_test[feature_columns].
    ↪ columns)

print(f"Training using {len(feature_columns)} HOBA features.")

# 6. Extract the target labels
y_train = df_train['is_fraud'].values
y_test = df_test['is_fraud'].values

print(f"Training features shape: {X_train_scaled.shape}")
print(f"Testing features shape: {X_test_scaled.shape}")

=====
Final HOBA Feature Count = 9
=====
Training using 9 HOBA features.
```

Training features shape: (1296675, 9)

Testing features shape: (555719, 9)

```
[10]: from sklearn.metrics import roc_curve, roc_auc_score, f1_score, \
      ↪ precision_score, recall_score
import numpy as np

def evaluate_fraud_business_metrics(y_true, y_pred_proba, max_fpr):
    # Calculate ROC curve
    fpr, tpr, thresholds = roc_curve(y_true, y_pred_proba)

    # Find the index of the largest FPR that is still under our max_fpr limit
    idx = np.where(fpr <= max_fpr)[0][-1]
    chosen_threshold = thresholds[idx]

    # Convert probabilities to binary predictions based on this new threshold
    y_pred_adj = (y_pred_proba >= chosen_threshold).astype(int)

    # Calculate final metrics
    f1 = f1_score(y_true, y_pred_adj)
    precision = precision_score(y_true, y_pred_adj)
    recall = recall_score(y_true, y_pred_adj)
    actual_fpr = fpr[idx]

    print(f"--- Metrics at Max FPR {max_fpr*100}% (Threshold: {chosen_threshold:
    ↪ .4f}) ---")
    print(f"Actual FPR: {actual_fpr*100:.4f}%")
    print(f"Recall (Fraud Caught): {recall*100:.2f}%")
    print(f"Precision (True Alarms): {precision*100:.2f}%")
    print(f"F1-Score: {f1:.4f}\n")
```

```
[11]: from sklearn.ensemble import RandomForestClassifier

# Initialize the model (limited depth and estimators to keep training time
    ↪ reasonable for a baseline)
print("Initializing and training Random Forest...")
rf_model = RandomForestClassifier(n_estimators=50, max_depth=15,
    ↪ class_weight='balanced', random_state=42, n_jobs=-1)

# Fit the model
rf_model.fit(X_train_scaled, y_train)
print("Training complete.")

# Predict probabilities on the testing set
y_pred_proba_rf = rf_model.predict_proba(X_test_scaled)[: , 1]

# 1. Overall AUC Score
```

```

auc_score = roc_auc_score(y_test, y_pred_proba_rf)
print(f"\nOverall ROC AUC Score: {auc_score:.4f}\n")

# 2. Evaluate at strict business tolerances (1% and 3% FPR)
evaluate_fraud_business_metrics(y_test, y_pred_proba_rf, max_fpr=0.01)
evaluate_fraud_business_metrics(y_test, y_pred_proba_rf, max_fpr=0.03)

```

Initializing and training Random Forest...
 Training complete.

Overall ROC AUC Score: 0.9843

--- Metrics at Max FPR 1.0% (Threshold: 0.5077) ---

Actual FPR: 0.9966%
 Recall (Fraud Caught): 86.34%
 Precision (True Alarms): 25.13%
 F1-Score: 0.3893

--- Metrics at Max FPR 3.0% (Threshold: 0.1840) ---

Actual FPR: 2.8838%
 Recall (Fraud Caught): 92.35%
 Precision (True Alarms): 11.04%
 F1-Score: 0.1972

```

[12]: import torch
import torch.nn as nn
import torch.nn.functional as F

class RBM(nn.Module):
    def __init__(self, visible_dim, hidden_dim):
        super(RBM, self).__init__()
        # Initialize weights and biases
        self.W = nn.Parameter(torch.randn(hidden_dim, visible_dim) * 0.01)
        self.h_bias = nn.Parameter(torch.zeros(hidden_dim))
        self.v_bias = nn.Parameter(torch.zeros(visible_dim))

    def sample_h(self, v):
        # Probability of hidden unit being active given visible units
        h_prob = torch.sigmoid(F.linear(v, self.W, self.h_bias))
        h_sample = torch.bernoulli(h_prob)
        return h_prob, h_sample

    def sample_v(self, h):
        # Probability of visible unit being active given hidden units
        v_prob = torch.sigmoid(F.linear(h, self.W.t(), self.v_bias))
        v_sample = torch.bernoulli(v_prob)

```

```

        return v_prob, v_sample

    def contrastive_divergence(self, v_pos, lr=0.01):
        # --- Positive Phase ---
        h_pos_prob, h_pos_sample = self.sample_h(v_pos)

        # --- Negative Phase (Gibbs Sampling) ---
        v_neg_prob, v_neg_sample = self.sample_v(h_pos_sample)
        h_neg_prob, _ = self.sample_h(v_neg_prob)

        # --- Calculate Gradients ---
        pos_grad = torch.matmul(h_pos_prob.t(), v_pos)
        neg_grad = torch.matmul(h_neg_prob.t(), v_neg_prob)

        # --- Update Weights & Biases ---
        batch_size = v_pos.size(0)
        self.W.data += lr * (pos_grad - neg_grad) / batch_size
        self.v_bias.data += lr * torch.mean(v_pos - v_neg_prob, dim=0)
        self.h_bias.data += lr * torch.mean(h_pos_prob - h_neg_prob, dim=0)

        # Calculate reconstruction error for monitoring
        error = torch.mean((v_pos - v_neg_prob)**2)
        return error.item()

```

```

[13]: from torch.utils.data import DataLoader, TensorDataset

# 1. Convert numpy arrays to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train_scaled)
y_train_tensor = torch.LongTensor(y_train)

X_test_tensor = torch.FloatTensor(X_test_scaled)

# 2. Create a DataLoader for batch processing
batch_size = 256
train_loader = DataLoader(TensorDataset(X_train_tensor), batch_size=batch_size,
    ↪shuffle=True)

# 3. Initialize the first RBM Layer
input_features = X_train_scaled.shape[1]
hidden_units_layer_1 = 64 # We compress the input features to extract core
    ↪patterns

rbm_layer_1 = RBM(visible_dim=input_features, hidden_dim=hidden_units_layer_1)

# 4. Train the RBM using Contrastive Divergence
epochs = 10
learning_rate = 0.01

```

```

print("Starting unsupervised RBM pre-training...")
for epoch in range(epochs):
    epoch_error = 0
    for batch in train_loader:
        v_batch = batch[0]
        # Update weights and get reconstruction error
        error = rbm_layer_1.contrastive_divergence(v_batch, lr=learning_rate)
        epoch_error += error

    avg_error = epoch_error / len(train_loader)
    print(f"Epoch {epoch+1}/{epochs} | Reconstruction Error: {avg_error:.4f}")

print("Layer 1 Pre-training Complete.")

```

```

Starting unsupervised RBM pre-training...
Epoch 1/10 | Reconstruction Error: 0.0162
Epoch 2/10 | Reconstruction Error: 0.0158
Epoch 3/10 | Reconstruction Error: 0.0161
Epoch 4/10 | Reconstruction Error: 0.0170
Epoch 5/10 | Reconstruction Error: 0.0181
Epoch 6/10 | Reconstruction Error: 0.0189
Epoch 7/10 | Reconstruction Error: 0.0196
Epoch 8/10 | Reconstruction Error: 0.0202
Epoch 9/10 | Reconstruction Error: 0.0206
Epoch 10/10 | Reconstruction Error: 0.0208
Layer 1 Pre-training Complete.

```

```

[14]: import torch.optim as optim

class SupervisedDBN(nn.Module):
    def __init__(self, pretrained_rbm):
        super(SupervisedDBN, self).__init__()
        self.hidden_layer = nn.Linear(pretrained_rbm.W.shape[1], pretrained_rbm.
→W.shape[0])
        self.hidden_layer.weight.data = pretrained_rbm.W.clone()
        self.hidden_layer.bias.data = pretrained_rbm.h_bias.clone()
        self.output_layer = nn.Linear(pretrained_rbm.W.shape[0], 1)

    def forward(self, x):
        x = torch.sigmoid(self.hidden_layer(x))
        # REMOVED the final torch.sigmoid() here because BCEWithLogitsLoss
→handles it
        x = self.output_layer(x)
        return x

dbn_model = SupervisedDBN(rbm_layer_1)

```

```

# --- Updated Training Loop ---

# Calculate the exact imbalance weight (Negative samples / Positive samples)
pos_weight_value = (len(y_train) - sum(y_train)) / sum(y_train)
class_weight = torch.FloatTensor([pos_weight_value])

# Use BCEWithLogitsLoss and apply the weight
criterion = nn.BCEWithLogitsLoss(pos_weight=class_weight)
optimizer = optim.Adam(dbn_model.parameters(), lr=0.001)

train_loader_supervised = DataLoader(TensorDataset(X_train_tensor,
↪ y_train_tensor.float()), batch_size=256, shuffle=True)
fine_tune_epochs = 15

print("Starting supervised fine-tuning with weighted loss...")
dbn_model.train()

for epoch in range(fine_tune_epochs):
    epoch_loss = 0
    for batch_features, batch_labels in train_loader_supervised:
        optimizer.zero_grad()
        outputs = dbn_model(batch_features).squeeze()
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    print(f"Epoch {epoch+1}/{fine_tune_epochs} | Classification Loss:↪
↪ {epoch_loss/len(train_loader_supervised):.4f}")

```

Starting supervised fine-tuning with weighted loss...

```

Epoch 1/15 | Classification Loss: 0.9616
Epoch 2/15 | Classification Loss: 0.5330
Epoch 3/15 | Classification Loss: 0.5005
Epoch 4/15 | Classification Loss: 0.4959
Epoch 5/15 | Classification Loss: 0.4924
Epoch 6/15 | Classification Loss: 0.4887
Epoch 7/15 | Classification Loss: 0.4891
Epoch 8/15 | Classification Loss: 0.4849
Epoch 9/15 | Classification Loss: 0.4835
Epoch 10/15 | Classification Loss: 0.4820
Epoch 11/15 | Classification Loss: 0.4812
Epoch 12/15 | Classification Loss: 0.4805
Epoch 13/15 | Classification Loss: 0.4794
Epoch 14/15 | Classification Loss: 0.4787
Epoch 15/15 | Classification Loss: 0.4784

```



```
[15]: dbn_model.eval()

with torch.no_grad():
    raw_logits = dbn_model(X_test_tensor).squeeze()
    # Apply sigmoid to convert logits back to probabilities (0 to 1)
    y_pred_proba_dbn_tensor = torch.sigmoid(raw_logits)

y_pred_proba_dbn = y_pred_proba_dbn_tensor.numpy()

auc_score_dbn = roc_auc_score(y_test, y_pred_proba_dbn)
print("\n--- Deep Belief Network (DBN) Performance ---")
print(f"Overall ROC AUC Score: {auc_score_dbn:.4f}\n")

evaluate_fraud_business_metrics(y_test, y_pred_proba_dbn, max_fpr=0.01)
evaluate_fraud_business_metrics(y_test, y_pred_proba_dbn, max_fpr=0.03)
```

```
--- Deep Belief Network (DBN) Performance ---
Overall ROC AUC Score: 0.9515
```

```
--- Metrics at Max FPR 1.0% (Threshold: 0.9403) ---
Actual FPR: 0.9876%
Recall (Fraud Caught): 75.20%
Precision (True Alarms): 22.78%
F1-Score: 0.3497
```

```
--- Metrics at Max FPR 3.0% (Threshold: 0.6690) ---
Actual FPR: 2.9958%
Recall (Fraud Caught): 83.96%
Precision (True Alarms): 9.80%
F1-Score: 0.1755
```

1 RBM LAYER 2

```
[16]: # 1. Extract the hidden representations from the first RBM
print("Extracting hidden features from Layer 1...")
with torch.no_grad():
    h1_features, _ = rbm_layer_1.sample_h(X_train_tensor)

# Create a DataLoader for the hidden features
train_loader_h1 = DataLoader(TensorDataset(h1_features), batch_size=batch_size,
    ↪shuffle=True)

# 2. Initialize the second RBM Layer
hidden_units_layer_2 = 32 # Further compressing the 64-dimensional features
    ↪down to 32
```

```

rbm_layer_2 = RBM(visible_dim=hidden_units_layer_1,
                  hidden_dim=hidden_units_layer_2)

# 3. Train RBM Layer 2
print("Starting unsupervised RBM Layer 2 pre-training...")
for epoch in range(epochs):
    epoch_error = 0
    for batch in train_loader_h1:
        v_batch = batch[0]
        error = rbm_layer_2.contrastive_divergence(v_batch, lr=learning_rate)
        epoch_error += error

    avg_error = epoch_error / len(train_loader_h1)
    print(f"Epoch {epoch+1}/{epochs} | Reconstruction Error: {avg_error:.4f}")

print("Layer 2 Pre-training Complete.")

```

```

Extracting hidden features from Layer 1...
Starting unsupervised RBM Layer 2 pre-training..
Epoch 1/10 | Reconstruction Error: 0.0008
Epoch 2/10 | Reconstruction Error: 0.0008
Epoch 3/10 | Reconstruction Error: 0.0008
Epoch 4/10 | Reconstruction Error: 0.0008
Epoch 5/10 | Reconstruction Error: 0.0008
Epoch 6/10 | Reconstruction Error: 0.0008
Epoch 7/10 | Reconstruction Error: 0.0009
Epoch 8/10 | Reconstruction Error: 0.0010
Epoch 9/10 | Reconstruction Error: 0.0010
Epoch 10/10 | Reconstruction Error: 0.0011
Layer 2 Pre-training Complete.

```

```

[17]: class DeepSupervisedDBN(nn.Module):
        def __init__(self, rbm1, rbm2):
            super(DeepSupervisedDBN, self).__init__()

            # Layer 1
            self.layer1 = nn.Linear(rbm1.W.shape[1], rbm1.W.shape[0])
            self.layer1.weight.data = rbm1.W.clone()
            self.layer1.bias.data = rbm1.h_bias.clone()

            # Layer 2
            self.layer2 = nn.Linear(rbm2.W.shape[1], rbm2.W.shape[0])
            self.layer2.weight.data = rbm2.W.clone()
            self.layer2.bias.data = rbm2.h_bias.clone()

            # Final Classification Layer
            self.output_layer = nn.Linear(rbm2.W.shape[0], 1)

```

```

def forward(self, x):
    x = torch.sigmoid(self.layer1(x))
    x = torch.sigmoid(self.layer2(x))
    x = self.output_layer(x) # Raw logits for BCEWithLogitsLoss
    return x

deep_dbn_model = DeepSupervisedDBN(rbm_layer_1, rbm_layer_2)
print("True Deep DBN architecture constructed.")

```

True Deep DBN architecture constructed.

```

[18]: # 1. Setup Optimizer and Loss
optimizer_deep = optim.Adam(deep_dbn_model.parameters(), lr=0.001)
criterion_deep = nn.BCEWithLogitsLoss(pos_weight=class_weight)

print("Starting deep supervised fine-tuning...")
deep_dbn_model.train()

for epoch in range(fine_tune_epochs):
    epoch_loss = 0
    for batch_features, batch_labels in train_loader_supervised:
        optimizer_deep.zero_grad()
        outputs = deep_dbn_model(batch_features).squeeze()
        loss = criterion_deep(outputs, batch_labels)
        loss.backward()
        optimizer_deep.step()
        epoch_loss += loss.item()

    print(f"Epoch {epoch+1}/{fine_tune_epochs} | Classification Loss:␣
↪{epoch_loss/len(train_loader_supervised):.4f}")

# 2. Evaluation
deep_dbn_model.eval()
with torch.no_grad():
    raw_logits_deep = deep_dbn_model(X_test_tensor).squeeze()
    y_pred_proba_deep = torch.sigmoid(raw_logits_deep).numpy()

auc_score_deep = roc_auc_score(y_test, y_pred_proba_deep)
print("\n--- Deep Belief Network (2-Layer) Performance ---")
print(f"Overall ROC AUC Score: {auc_score_deep:.4f}\n")

evaluate_fraud_business_metrics(y_test, y_pred_proba_deep, max_fpr=0.01)
evaluate_fraud_business_metrics(y_test, y_pred_proba_deep, max_fpr=0.03)

```

Starting deep supervised fine-tuning..

Epoch 1/15 | Classification Loss: 1.0013

Epoch 2/15 | Classification Loss: 0.5092

```
Epoch 3/15 | Classification Loss: 0.4907
Epoch 4/15 | Classification Loss: 0.4857
Epoch 5/15 | Classification Loss: 0.4806
Epoch 6/15 | Classification Loss: 0.4778
Epoch 7/15 | Classification Loss: 0.4756
Epoch 8/15 | Classification Loss: 0.4734
Epoch 9/15 | Classification Loss: 0.4708
Epoch 10/15 | Classification Loss: 0.4722
Epoch 11/15 | Classification Loss: 0.4700
Epoch 12/15 | Classification Loss: 0.4664
Epoch 13/15 | Classification Loss: 0.4663
Epoch 14/15 | Classification Loss: 0.4652
Epoch 15/15 | Classification Loss: 0.4650
```

```
--- Deep Belief Network (2-Layer) Performance ---
Overall ROC AUC Score: 0.9516
```

```
--- Metrics at Max FPR 1.0% (Threshold: 0.9618) ---
Actual FPR: 0.9793%
Recall (Fraud Caught): 75.10%
Precision (True Alarms): 22.91%
F1-Score: 0.3511
```

```
--- Metrics at Max FPR 3.0% (Threshold: 0.7916) ---
Actual FPR: 2.9902%
Recall (Fraud Caught): 84.10%
Precision (True Alarms): 9.83%
F1-Score: 0.1760
```

```
[19]: # Assuming df is your chronologically sorted, combined train/test dataframe
df_time_indexed = df.set_index('trans_date_trans_time').sort_index()

# 1. Define the HOBA Methodology Parameters
time_windows = ['1D', '3D', '7D', '15D']
categories = df['category'].unique()

print("Expanding HOBA Feature Matrix...")

# 2. Automate the Transaction Aggregation Strategy
for window in time_windows:
    # A. Time-based general aggregations
    df[f'HOBA_amt_sum_{window}'] = df_time_indexed.groupby('cc_num')['amt'].
    ↪transform(lambda x: x.rolling(window).sum()).values
    df[f'HOBA_amt_max_{window}'] = df_time_indexed.groupby('cc_num')['amt'].
    ↪transform(lambda x: x.rolling(window).max()).values
```

```

# B. Category-specific aggregations (Homogeneity-Oriented)
for cat in categories:
    # Create a temporary column that only has amounts for this specific
    ↪category (0 otherwise)
    df_time_indexed['temp_amt'] = np.where(df_time_indexed['category'] ==
    ↪cat, df_time_indexed['amt'], 0)

    # Calculate rolling sum for this specific category over the time window
    df[f'HOB_A_amt_sum_{cat}_{window}'] = df_time_indexed.
    ↪groupby('cc_num')['temp_amt'].transform(
        lambda x: x.rolling(window).sum()
    ).values

# 3. Clean up temporary columns
df_time_indexed.drop(columns=['temp_amt'], inplace=True, errors='ignore')

print(f"Feature expansion complete. New dataset shape: {df.shape}")

```

Expanding HOBA Feature Matrix...

Feature expansion complete. New dataset shape: (1852394, 94)

```

[20]: from sklearn.preprocessing import MinMaxScaler
import torch
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# --- 1. Split, Isolate, and Scale the 94 Features ---
df_train = df[df['is_test_set'] == 0].copy()
df_test = df[df['is_test_set'] == 1].copy()

columns_to_drop = [
    'is_fraud', 'is_test_set', 'cc_num', 'trans_num', 'merchant', 'category',
    'first', 'last', 'gender', 'street', 'city', 'state', 'job', 'dob',
    'trans_date_trans_time', 'unix_time', 'lat', 'long', 'merch_lat',
    ↪'merch_long',
    'Unnamed: 0'
]

feature_columns = FINAL_HOBA_FEATURES

print(f"Using frozen HOBA feature set ({len(feature_columns)} features)")

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(df_train[feature_columns])
X_test_scaled = scaler.transform(df_test[feature_columns])

X_train_tensor = torch.FloatTensor(X_train_scaled)

```

```

X_test_tensor = torch.FloatTensor(X_test_scaled)
train_loader = DataLoader(TensorDataset(X_train_tensor), batch_size=256,
    ↪shuffle=True)

# --- 2. Unsupervised Pre-Training (Layer 1 & 2) ---
input_features = len(feature_columns)
rbm_layer_1 = RBM(visible_dim=input_features, hidden_dim=64)

print("\n[Phase 1] Pre-training RBM Layer 1...")
for epoch in range(10):
    for batch in train_loader:
        rbm_layer_1.contrastive_divergence(batch[0], lr=0.01)

print("[Phase 2] Extracting hidden features and pre-training RBM Layer 2...")
with torch.no_grad():
    h1_features, _ = rbm_layer_1.sample_h(X_train_tensor)
train_loader_h1 = DataLoader(TensorDataset(h1_features), batch_size=256,
    ↪shuffle=True)

rbm_layer_2 = RBM(visible_dim=64, hidden_dim=32)
for epoch in range(10):
    for batch in train_loader_h1:
        rbm_layer_2.contrastive_divergence(batch[0], lr=0.01)

# --- 3. Deep Supervised Fine-Tuning ---
print("\n[Phase 3] Supervised Fine-Tuning of the Deep Architecture...")
deep_dbn_model = DeepSupervisedDBN(rbm_layer_1, rbm_layer_2)

# Applying the same class weight logic as before
pos_weight_value = (len(y_train) - sum(y_train)) / sum(y_train)
class_weight = torch.FloatTensor([pos_weight_value])

optimizer_deep = optim.Adam(deep_dbn_model.parameters(), lr=0.001)
criterion_deep = nn.BCEWithLogitsLoss(pos_weight=class_weight)
train_loader_supervised = DataLoader(TensorDataset(X_train_tensor, torch.
    ↪LongTensor(y_train).float()), batch_size=256, shuffle=True)

deep_dbn_model.train()
for epoch in range(15):
    for batch_features, batch_labels in train_loader_supervised:
        optimizer_deep.zero_grad()
        outputs = deep_dbn_model(batch_features).squeeze()
        loss = criterion_deep(outputs, batch_labels)
        loss.backward()
        optimizer_deep.step()

# --- 4. Final Evaluation ---

```

```

deep_dbn_model.eval()
with torch.no_grad():
    raw_logits_deep = deep_dbn_model(X_test_tensor).squeeze()
    y_pred_proba_deep = torch.sigmoid(raw_logits_deep).numpy()

auc_score_deep = roc_auc_score(y_test, y_pred_proba_deep)
print("\n=====")
print("          FINAL DEEP BELIEF NETWORK PERFORMANCE          ")
print("=====")
print(f"Overall ROC AUC Score: {auc_score_deep:.4f}\n")

evaluate_fraud_business_metrics(y_test, y_pred_proba_deep, max_fpr=0.01)
evaluate_fraud_business_metrics(y_test, y_pred_proba_deep, max_fpr=0.03)

```

Using frozen HOBA feature set (9 features)

[Phase 1] Pre-training RBM Layer 1...

[Phase 2] Extracting hidden features and pre-training RBM Layer 2...

[Phase 3] Supervised Fine-Tuning of the Deep Architecture...

```

=====
          FINAL DEEP BELIEF NETWORK PERFORMANCE
=====
Overall ROC AUC Score: 0.9517

```

```

--- Metrics at Max FPR 1.0% (Threshold: 0.9646) ---
Actual FPR: 0.9991%
Recall (Fraud Caught): 75.20%
Precision (True Alarms): 22.58%
F1-Score: 0.3473

```

```

--- Metrics at Max FPR 3.0% (Threshold: 0.8187) ---
Actual FPR: 2.9954%
Recall (Fraud Caught): 84.06%
Precision (True Alarms): 9.81%
F1-Score: 0.1756

```

```

[21]: import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

class FraudCNN(nn.Module):
    def __init__(self, num_features):
        super(FraudCNN, self).__init__()
        # 1D Convolution: input channels=1, output channels=16, kernel_size=3

```

```

        self.conv1 = nn.Conv1d(in_channels=1, out_channels=16, kernel_size=3,
↪padding=1)
        self.pool = nn.MaxPool1d(kernel_size=2)

        # Calculate the flattened dimension after pooling
        self.flattened_dim = 16 * (num_features // 2)

        # Fully connected classification layers
        self.fc1 = nn.Linear(self.flattened_dim, 32)
        self.fc2 = nn.Linear(32, 1)

    def forward(self, x):
        # CNN expects input shape: (batch_size, channels, features)
        # We add a channel dimension to our 1D features
        x = x.unsqueeze(1)

        x = self.pool(F.relu(self.conv1(x)))
        x = x.view(-1, self.flattened_dim) # Flatten
        x = F.relu(self.fc1(x))
        x = self.fc2(x) # Raw logits
        return x

print("CNN Architecture Initialized.")

```

CNN Architecture Initialized.

```

[22]: class ElmanRNN(nn.Module):
        def __init__(self, input_size, hidden_size=64):
            super(ElmanRNN, self).__init__()
            # PyTorch's nn.RNN is the standard Elman network architecture
            self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_size,
↪batch_first=True)

            # Final classification layer
            self.fc = nn.Linear(hidden_size, 1)

        def forward(self, x):
            # RNN expects input shape: (batch_size, sequence_length, features)
            # We treat the HOBA feature vector as a sequence of 1 time-step
↪containing all behavioral data
            x = x.unsqueeze(1)

            # Out contains the output features from the last layer of the RNN
            out, hidden = self.rnn(x)

            # Take the output of the final time step
            out = out[:, -1, :]

```



```

        out = self.fc(out) # Raw logits
        return out

print("Elman RNN Architecture Initialized.")

```

Elman RNN Architecture Initialized.

```

[23]: # --- Setup ---
num_features = X_train_tensor.shape[1]
cnn_model = FraudCNN(num_features)
rnn_model = ElmanRNN(num_features)

# Use the class weight calculated earlier
criterion = nn.BCEWithLogitsLoss(pos_weight=class_weight)
optimizer_cnn = optim.Adam(cnn_model.parameters(), lr=0.001)
optimizer_rnn = optim.Adam(rnn_model.parameters(), lr=0.001)

epochs = 15

# --- Train CNN ---
print("\n--- Training Convolutional Neural Network (CNN) ---")
cnn_model.train()
for epoch in range(epochs):
    epoch_loss = 0
    for batch_features, batch_labels in train_loader_supervised:
        optimizer_cnn.zero_grad()
        outputs = cnn_model(batch_features).squeeze()
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer_cnn.step()
        epoch_loss += loss.item()
    print(f"Epoch {epoch+1}/{epochs} | Loss: {epoch_loss/len(train_loader_supervised):.4f}")

# --- Train RNN ---
print("\n--- Training Elman Recurrent Neural Network (RNN) ---")
rnn_model.train()
for epoch in range(epochs):
    epoch_loss = 0
    for batch_features, batch_labels in train_loader_supervised:
        optimizer_rnn.zero_grad()
        outputs = rnn_model(batch_features).squeeze()
        loss = criterion(outputs, batch_labels)
        loss.backward()
        optimizer_rnn.step()
        epoch_loss += loss.item()

```

```

    print(f"Epoch {epoch+1}/{epochs} | Loss: {epoch_loss/
↪len(train_loader_supervised):.4f}")

# --- Evaluate Both Models ---
cnn_model.eval()
rnn_model.eval()

with torch.no_grad():
    y_pred_proba_cnn = torch.sigmoid(cnn_model(X_test_tensor).squeeze()).numpy()
    y_pred_proba_rnn = torch.sigmoid(rnn_model(X_test_tensor).squeeze()).numpy()

print("\n=====")
print("          CNN PERFORMANCE BENCHMARK          ")
print("=====")
print(f"Overall ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_cnn):.4f}\n")
evaluate_fraud_business_metrics(y_test, y_pred_proba_cnn, max_fpr=0.01)

print("\n=====")
print("          RNN PERFORMANCE BENCHMARK          ")
print("=====")
print(f"Overall ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_rnn):.4f}\n")
evaluate_fraud_business_metrics(y_test, y_pred_proba_rnn, max_fpr=0.01)

```

--- Training Convolutional Neural Network (CNN) ---

```

Epoch 1/15 | Loss: 0.6394
Epoch 2/15 | Loss: 0.4850
Epoch 3/15 | Loss: 0.4791
Epoch 4/15 | Loss: 0.4714
Epoch 5/15 | Loss: 0.4640
Epoch 6/15 | Loss: 0.4562
Epoch 7/15 | Loss: 0.4513
Epoch 8/15 | Loss: 0.4437
Epoch 9/15 | Loss: 0.4412
Epoch 10/15 | Loss: 0.4382
Epoch 11/15 | Loss: 0.4358
Epoch 12/15 | Loss: 0.4353
Epoch 13/15 | Loss: 0.4310
Epoch 14/15 | Loss: 0.4287
Epoch 15/15 | Loss: 0.4251

```

--- Training Elman Recurrent Neural Network (RNN) ---

```

Epoch 1/15 | Loss: 0.6224
Epoch 2/15 | Loss: 0.5060
Epoch 3/15 | Loss: 0.4983
Epoch 4/15 | Loss: 0.4947
Epoch 5/15 | Loss: 0.4920
Epoch 6/15 | Loss: 0.4910

```

```
Epoch 7/15 | Loss: 0.4909
Epoch 8/15 | Loss: 0.4886
Epoch 9/15 | Loss: 0.4869
Epoch 10/15 | Loss: 0.4853
Epoch 11/15 | Loss: 0.4846
Epoch 12/15 | Loss: 0.4835
Epoch 13/15 | Loss: 0.4839
Epoch 14/15 | Loss: 0.4829
Epoch 15/15 | Loss: 0.4810
```

```
=====
                        CNN PERFORMANCE BENCHMARK
=====
```

```
Overall ROC AUC Score: 0.9549
```

```
--- Metrics at Max FPR 1.0% (Threshold: 0.8004) ---
Actual FPR: 0.9964%
Recall (Fraud Caught): 77.44%
Precision (True Alarms): 23.14%
F1-Score: 0.3564
```

```
=====
                        RNN PERFORMANCE BENCHMARK
=====
```

```
Overall ROC AUC Score: 0.9518
```

```
--- Metrics at Max FPR 1.0% (Threshold: 0.9367) ---
Actual FPR: 0.9991%
Recall (Fraud Caught): 75.62%
Precision (True Alarms): 22.68%
F1-Score: 0.3489
```

```
[24]: print("Engineering Rule-Based Features...")

# 1. Abnormal Time Flag (1 AM to 5 AM)
df['hour'] = df['trans_date_trans_time'].dt.hour
df['is_abnormal_time'] = df['hour'].apply(lambda x: 1 if 1 <= x <= 5 else 0)

# 2. High-Value Transaction Flag
df['is_high_amount'] = (df['amt'] > 500).astype(int)

# 3. High-Velocity Flag (More than 3 transactions in the last hour)
df_time_indexed = df.set_index('trans_date_trans_time').sort_index()
df['tx_count_1h'] = df_time_indexed.groupby('cc_num')['amt'].transform(lambda x:
    ↪ x.rolling('1h').count()).values
```

```

df['is_high_velocity'] = (df['tx_count_1h'] > 3).astype(int)

# Clean up temporary hour column
df.drop(columns=['hour'], inplace=True, errors='ignore')

print(f"Rule-based features added. Total columns: {df.shape[1]}")

```

Engineering Rule-Based Features...

Rule-based features added. Total columns: 98

```

[25]: from sklearn.svm import SVC
      from sklearn.neural_network import MLPClassifier
      from sklearn.utils import resample

      # --- 1. Re-Split and Scale the Final Dataset ---
      df_train = df[df['is_test_set'] == 0].copy()
      df_test = df[df['is_test_set'] == 1].copy()

      feature_columns = FINAL_HOBA_FEATURES

      print(f"Using frozen HOBA feature set ({len(feature_columns)} features)")

      X_train_scaled = scaler.fit_transform(df_train[feature_columns])
      X_test_scaled = scaler.transform(df_test[feature_columns])

      # --- 2. Train the BPNN (Shallow Neural Network) ---
      print("\n--- Training BPNN (Shallow Neural Network) ---")
      # Using a single hidden layer to represent a shallow BPNN
      bpnn_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=20,
      ↪ random_state=42)
      bpnn_model.fit(X_train_scaled, y_train)

      y_pred_proba_bpnn = bpnn_model.predict_proba(X_test_scaled)[: , 1]

      print("\n=====")
      print("          BPNN PERFORMANCE BENCHMARK          ")
      print("=====")
      print(f"Overall ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_bpnn):.
      ↪ 4f}\n")
      evaluate_fraud_business_metrics(y_test, y_pred_proba_bpnn, max_fpr=0.01)

      # --- 3. Downsample and Train the SVM ---
      print("\n--- Downsampling data and Training SVM ---")
      # Combine X and y to downsample proportionally
      train_combined = np.column_stack((X_train_scaled, y_train))

      # Separate majority and minority classes

```

```

fraud = train_combined[train_combined[:, -1] == 1]
legit = train_combined[train_combined[:, -1] == 0]

# Downsample legitimate transactions heavily to allow the SVM to run in
↳minutes, not days
legit_downsampled = resample(legit, replace=False, n_samples=len(fraud) * 10,
↳random_state=42)

# Combine and shuffle
downsampled_train = np.vstack((fraud, legit_downsampled))
np.random.shuffle(downsampled_train)

X_train_svm = downsampled_train[:, :-1]
y_train_svm = downsampled_train[:, -1]

# Initialize and train the Support Vector Machine
svm_model = SVC(kernel='rbf', probability=True, random_state=42,
↳class_weight='balanced')
svm_model.fit(X_train_svm, y_train_svm)

y_pred_proba_svm = svm_model.predict_proba(X_test_scaled)[: , 1]

print("\n=====")
print("          SVM PERFORMANCE BENCHMARK          ")
print("=====")
print(f"Overall ROC AUC Score: {roc_auc_score(y_test, y_pred_proba_svm):.4f}\n")
evaluate_fraud_business_metrics(y_test, y_pred_proba_svm, max_fpr=0.01)

```

Using frozen HOBA feature set (9 features)

--- Training BPNN (Shallow Neural Network) ---

```

=====
                BPNN PERFORMANCE BENCHMARK
=====
Overall ROC AUC Score: 0.9533

```

--- Metrics at Max FPR 1.0% (Threshold: 0.0459) ---

```

Actual FPR: 0.9984%
Recall (Fraud Caught): 77.34%
Precision (True Alarms): 23.09%
F1-Score: 0.3556

```

--- Downsampling data and Training SVM ---

```

=====

```

SVM PERFORMANCE BENCHMARK

Overall ROC AUC Score: 0.9518

--- Metrics at Max FPR 1.0% (Threshold: 0.4312) ---

Actual FPR: 0.9988%

Recall (Fraud Caught): 74.41%

Precision (True Alarms): 22.40%

F1-Score: 0.3443

```
[28]: import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.utils import resample
from sklearn.metrics import accuracy_score, roc_curve, f1_score,
    precision_score, recall_score
import warnings
warnings.filterwarnings('ignore')

print("Defining Dynamic Architectures...")

class FraudCNN(nn.Module):
    def __init__(self, num_features):
        super(FraudCNN, self).__init__()
        self.conv1 = nn.Conv1d(in_channels=1, out_channels=16, kernel_size=3,
    padding=1)
        self.pool = nn.MaxPool1d(kernel_size=2)
        self.flattened_dim = 16 * (num_features // 2)
        self.fc1 = nn.Linear(self.flattened_dim, 32)
        self.fc2 = nn.Linear(32, 1)
    def forward(self, x):
        x = x.unsqueeze(1)
        x = self.pool(torch.relu(self.conv1(x)))
        x = x.view(-1, self.flattened_dim)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

class ElmanRNN(nn.Module):
```

```

def __init__(self, input_size, hidden_size=32):
    super(ElmanRNN, self).__init__()
    self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_size,
↳batch_first=True)
    self.fc = nn.Linear(hidden_size, 1)
def forward(self, x):
    x = x.unsqueeze(1)
    out, _ = self.rnn(x)
    return self.fc(out[:, -1, :])

class RBM(nn.Module):
    def __init__(self, visible_dim, hidden_dim):
        super(RBM, self).__init__()
        self.W = nn.Parameter(torch.randn(hidden_dim, visible_dim) * 0.01)
        self.h_bias = nn.Parameter(torch.zeros(hidden_dim))
        self.v_bias = nn.Parameter(torch.zeros(visible_dim))
    def sample_h(self, v):
        h_prob = torch.sigmoid(F.linear(v, self.W, self.h_bias))
        return h_prob, torch.bernoulli(h_prob)
    def sample_v(self, h):
        v_prob = torch.sigmoid(F.linear(h, self.W.t(), self.v_bias))
        return v_prob, torch.bernoulli(v_prob)
    def contrastive_divergence(self, v_pos, lr=0.01):
        h_pos_prob, h_pos_sample = self.sample_h(v_pos)
        v_neg_prob, _ = self.sample_v(h_pos_sample)
        h_neg_prob, _ = self.sample_h(v_neg_prob)
        pos_grad = torch.matmul(h_pos_prob.t(), v_pos)
        neg_grad = torch.matmul(h_neg_prob.t(), v_neg_prob)
        self.W.data += lr * (pos_grad - neg_grad) / v_pos.size(0)
        self.v_bias.data += lr * torch.mean(v_pos - v_neg_prob, dim=0)
        self.h_bias.data += lr * torch.mean(h_pos_prob - h_neg_prob, dim=0)

class DeepSupervisedDBN(nn.Module):
    def __init__(self, rbm1, rbm2):
        super(DeepSupervisedDBN, self).__init__()
        self.layer1 = nn.Linear(rbm1.W.shape[1], rbm1.W.shape[0])
        self.layer1.weight.data, self.layer1.bias.data = rbm1.W.clone(), rbm1.
↳h_bias.clone()
        self.layer2 = nn.Linear(rbm2.W.shape[1], rbm2.W.shape[0])
        self.layer2.weight.data, self.layer2.bias.data = rbm2.W.clone(), rbm2.
↳h_bias.clone()
        self.output_layer = nn.Linear(rbm2.W.shape[0], 1)
    def forward(self, x):
        x = torch.sigmoid(self.layer1(x))
        x = torch.sigmoid(self.layer2(x))
        return self.output_layer(x)

```

```

# Extract targets and class weights
y_train, y_test = df_train['is_fraud'].values, df_test['is_fraud'].values
pos_weight_value = (len(y_train) - sum(y_train)) / sum(y_train)
criterion = nn.BCEWithLogitsLoss(pos_weight=torch.
    ↪FloatTensor([pos_weight_value]))

def get_table_metrics(y_true, y_pred_proba, max_fpr):
    fpr, tpr, thresholds = roc_curve(y_true, y_pred_proba)
    idx = np.where(fpr <= max_fpr)[0][-1]
    chosen_threshold = thresholds[idx]
    y_pred_adj = (y_pred_proba >= chosen_threshold).astype(int)
    return {
        'FPR': f"{max_fpr*100:.0f}%",
        'F1-Measure': f"{f1_score(y_true, y_pred_adj):.3f}",
        'Precision': f"{precision_score(y_true, y_pred_adj)*100:.2f}%",
        'Recall': f"{recall_score(y_true, y_pred_adj)*100:.2f}%",
        'Accuracy': f"{accuracy_score(y_true, y_pred_adj)*100:.2f}%"
    }

print("Setup complete.")

```

Defining Dynamic Architectures...
Setup complete.

```

[29]: print("--- STARTING HOBA PIPELINE ---")
columns_to_drop = ['is_fraud', 'is_test_set', 'cc_num', 'trans_num',
    ↪'merchant', 'category', 'first', 'last', 'gender', 'street', 'city',
    ↪'state', 'job', 'dob', 'trans_date_trans_time', 'unix_time', 'lat', 'long',
    ↪'merch_lat', 'merch_long', 'Unnamed: 0']
hoba_cols = [col for col in df_train.columns if col not in columns_to_drop]

scaler_hoba = MinMaxScaler()
X_tr_hoba = scaler_hoba.fit_transform(df_train[hoba_cols])
X_te_hoba = scaler_hoba.transform(df_test[hoba_cols])

X_tr_hoba_t = torch.FloatTensor(X_tr_hoba)
X_te_hoba_t = torch.FloatTensor(X_te_hoba)
train_loader_hoba = DataLoader(TensorDataset(X_tr_hoba_t, torch.
    ↪LongTensor(y_train).float()), batch_size=256, shuffle=True)

# 1. Train Traditional ML Models
print("Training BPNN (HOBA)...")
bpnn_h = MLPClassifier(hidden_layer_sizes=(50,), max_iter=15, random_state=42).
    ↪fit(X_tr_hoba, y_train)

print("Training RF (HOBA)...")

```



```

rf_h = RandomForestClassifier(n_estimators=30, max_depth=10,
    ↪class_weight='balanced', random_state=42, n_jobs=-1).fit(X_tr_hoba, y_train)

print("Training SVM (HOBA)...")
legit_down_h = resample(X_tr_hoba[y_train==0], replace=False,
    ↪n_samples=sum(y_train)*5, random_state=42)
X_svm_h = np.vstack((X_tr_hoba[y_train==1], legit_down_h))
y_svm_h = np.hstack((np.ones(sum(y_train)), np.zeros(len(legit_down_h))))
svm_h = SVC(kernel='rbf', probability=True, random_state=42,
    ↪class_weight='balanced').fit(X_svm_h, y_svm_h)

# 2. Train Deep Learning Models
print(f"Training CNN and RNN (HOBA) on {len(hoba_cols)} features...")
cnn_h, rnn_h = FraudCNN(len(hoba_cols)), ElmanRNN(len(hoba_cols))
opt_cnn_h, opt_rnn_h = optim.Adam(cnn_h.parameters(), lr=0.005), optim.
    ↪Adam(rnn_h.parameters(), lr=0.005)

for _ in range(3):
    for b_x, b_y in train_loader_hoba:
        opt_cnn_h.zero_grad(); loss1 = criterion(cnn_h(b_x).squeeze(), b_y);
        ↪loss1.backward(); opt_cnn_h.step()
        opt_rnn_h.zero_grad(); loss2 = criterion(rnn_h(b_x).squeeze(), b_y);
        ↪loss2.backward(); opt_rnn_h.step()

print("Training DBN (HOBA)...")
rbm1_h = RBM(len(hoba_cols), 64)
for _ in range(2):
    for b in train_loader_hoba: rbm1_h.contrastive_divergence(b[0])
with torch.no_grad(): h1_h, _ = rbm1_h.sample_h(X_tr_hoba_t)
loader_h1_h = DataLoader(TensorDataset(h1_h), batch_size=256, shuffle=True)

rbm2_h = RBM(64, 32)
for _ in range(2):
    for b in loader_h1_h: rbm2_h.contrastive_divergence(b[0])

dbn_h = DeepSupervisedDBN(rbm1_h, rbm2_h)
opt_dbn_h = optim.Adam(dbn_h.parameters(), lr=0.005)
for _ in range(3):
    for b_x, b_y in train_loader_hoba:
        opt_dbn_h.zero_grad(); loss = criterion(dbn_h(b_x).squeeze(), b_y);
        ↪loss.backward(); opt_dbn_h.step()

# 3. Extract Predictions and Generate Tables
print("Extracting HOBA Predictions...")
with torch.no_grad():
    hoba_predictions = {

```

```

        'BPNN': bpnn_h.predict_proba(X_te_hoba)[: , 1],
        'SVM': svm_h.predict_proba(X_te_hoba)[: , 1],
        'RF': rf_h.predict_proba(X_te_hoba)[: , 1],
        'CNN': torch.sigmoid(cnn_h(X_te_hoba_t).squeeze()).numpy(),
        'RNN': torch.sigmoid(rnn_h(X_te_hoba_t).squeeze()).numpy(),
        'DBN': torch.sigmoid(dbn_h(X_te_hoba_t).squeeze()).numpy()
    }

print("\n--- Table 5a: HOBA Features (1% FPR) ---")
display(pd.DataFrame([dict({'Classifiers': name}, **get_table_metrics(y_test,
    ↪probas, 0.01)) for name, probas in hoba_predictions.items()])).
    ↪set_index('Classifiers')[['FPR', 'F1-Measure', 'Precision', 'Recall',
    ↪'Accuracy']]

print("\n--- Table 5b: HOBA Features (3% FPR) ---")
display(pd.DataFrame([dict({'Classifiers': name}, **get_table_metrics(y_test,
    ↪probas, 0.03)) for name, probas in hoba_predictions.items()])).
    ↪set_index('Classifiers')[['FPR', 'F1-Measure', 'Precision', 'Recall',
    ↪'Accuracy']]

```

--- STARTING HOBA PIPELINE ---

Training BPNN (HOBA)...

Training RF (HOBA)...

Training SVM (HOBA)...

Training CNN and RNN (HOBA) on 77 features...

Training DBN (HOBA)...

Extracting HOBA Predictions...

--- Table 5a: HOBA Features (1% FPR) ---

	FPR	F1-Measure	Precision	Recall	Accuracy
Classifiers					
BPNN	1%	0.360	23.33%	78.37%	98.92%
SVM	1%	0.338	22.01%	72.45%	98.90%
RF	1%	0.375	24.28%	82.56%	98.94%
CNN	1%	0.341	22.23%	73.66%	98.90%
RNN	1%	0.346	22.50%	74.87%	98.91%
DBN	1%	0.346	22.49%	74.83%	98.91%

--- Table 5b: HOBA Features (3% FPR) ---

	FPR	F1-Measure	Precision	Recall	Accuracy
Classifiers					
BPNN	3%	0.181	10.09%	86.34%	96.98%
SVM	3%	0.176	9.81%	83.36%	96.98%
RF	3%	0.188	10.51%	90.68%	96.99%
CNN	3%	0.176	9.84%	84.15%	96.96%
RNN	3%	0.178	9.94%	84.85%	96.98%

DBN 3% 0.178 9.96% 85.17% 96.97%

```
[30]: import matplotlib.pyplot as plt

print("--- STARTING RFM PIPELINE ---")
rfm_cols = ['amt', 'zip', 'city_pop', 'distance_to_merchant_km']

scaler_rfm = MinMaxScaler()
X_tr_rfm = scaler_rfm.fit_transform(df_train[rfm_cols])
X_te_rfm = scaler_rfm.transform(df_test[rfm_cols])

X_tr_rfm_t = torch.FloatTensor(X_tr_rfm)
X_te_rfm_t = torch.FloatTensor(X_te_rfm)
train_loader_rfm = DataLoader(TensorDataset(X_tr_rfm_t, torch.
    ↳ LongTensor(y_train).float()), batch_size=256, shuffle=True)

# 1. Train Traditional ML Models
print("Training BPNN (RFM)...")
bpnn_r = MLPClassifier(hidden_layer_sizes=(50,), max_iter=15, random_state=42).
    ↳ fit(X_tr_rfm, y_train)

print("Training RF (RFM)...")
rf_r = RandomForestClassifier(n_estimators=30, max_depth=10,
    ↳ class_weight='balanced', random_state=42, n_jobs=-1).fit(X_tr_rfm, y_train)

print("Training SVM (RFM)...")
legit_down_r = resample(X_tr_rfm[y_train==0], replace=False,
    ↳ n_samples=sum(y_train)*5, random_state=42)
X_svm_r = np.vstack((X_tr_rfm[y_train==1], legit_down_r))
y_svm_r = np.hstack((np.ones(sum(y_train)), np.zeros(len(legit_down_r))))
svm_r = SVC(kernel='rbf', probability=True, random_state=42,
    ↳ class_weight='balanced').fit(X_svm_r, y_svm_r)

# 2. Train Deep Learning Models
print(f"Training CNN and RNN (RFM) on {len(rfm_cols)} features...")
cnn_r, rnn_r = FraudCNN(len(rfm_cols)), ElmanRNN(len(rfm_cols))
opt_cnn_r, opt_rnn_r = optim.Adam(cnn_r.parameters(), lr=0.005), optim.
    ↳ Adam(rnn_r.parameters(), lr=0.005)

for _ in range(3):
    for b_x, b_y in train_loader_rfm:
        opt_cnn_r.zero_grad(); loss1 = criterion(cnn_r(b_x).squeeze(), b_y);
        ↳ loss1.backward(); opt_cnn_r.step()
        opt_rnn_r.zero_grad(); loss2 = criterion(rnn_r(b_x).squeeze(), b_y);
        ↳ loss2.backward(); opt_rnn_r.step()

print("Training DBN (RFM)...")
```

```

rbm1_r = RBM(len(rfm_cols), 32)
for _ in range(2):
    for b in train_loader_rfm: rbm1_r.contrastive_divergence(b[0])
with torch.no_grad(): h1_r, _ = rbm1_r.sample_h(X_tr_rfm_t)
loader_h1_r = DataLoader(TensorDataset(h1_r), batch_size=256, shuffle=True)

rbm2_r = RBM(32, 16)
for _ in range(2):
    for b in loader_h1_r: rbm2_r.contrastive_divergence(b[0])

dbn_r = DeepSupervisedDBN(rbm1_r, rbm2_r)
opt_dbn_r = optim.Adam(dbn_r.parameters(), lr=0.005)
for _ in range(3):
    for b_x, b_y in train_loader_rfm:
        opt_dbn_r.zero_grad(); loss = criterion(dbn_r(b_x).squeeze(), b_y);
        ↪ loss.backward(); opt_dbn_r.step()

# 3. Extract Predictions and Generate Tables
print("Extracting RFM Predictions...")
with torch.no_grad():
    rfm_predictions = {
        'BPNN': bpnn_r.predict_proba(X_te_rfm)[: , 1],
        'SVM': svm_r.predict_proba(X_te_rfm)[: , 1],
        'RF': rf_r.predict_proba(X_te_rfm)[: , 1],
        'CNN': torch.sigmoid(cnn_r(X_te_rfm_t).squeeze()).numpy(),
        'RNN': torch.sigmoid(rnn_r(X_te_rfm_t).squeeze()).numpy(),
        'DBN': torch.sigmoid(dbn_r(X_te_rfm_t).squeeze()).numpy()
    }

print("\n--- Table 5a: RFM Features (1% FPR) ---")
display(pd.DataFrame([dict({'Classifiers': name}, **get_table_metrics(y_test,
    ↪ probas, 0.01)) for name, probas in rfm_predictions.items()])).
    ↪ set_index('Classifiers')[['FPR', 'F1-Measure', 'Precision', 'Recall',
    ↪ 'Accuracy']])

print("\n--- Table 5b: RFM Features (3% FPR) ---")
display(pd.DataFrame([dict({'Classifiers': name}, **get_table_metrics(y_test,
    ↪ probas, 0.03)) for name, probas in rfm_predictions.items()])).
    ↪ set_index('Classifiers')[['FPR', 'F1-Measure', 'Precision', 'Recall',
    ↪ 'Accuracy']])

# --- 4. Draw the Final ROC Curves ---
print("\nDrawing Final ROC Curves with DBN...")
fig, axes = plt.subplots(2, 2, figsize=(16, 16))
plt.style.use('default')

```

```

colors = {'DBN': 'red', 'CNN': 'green', 'RNN': 'blue', 'RF': 'cyan', 'SVM': 'magenta', 'BPNN': 'black'}

def plot_roc(ax, y_true, y_pred, label, color, linestyle='-'):
    fpr, tpr, _ = roc_curve(y_true, y_pred)
    ax.plot(fpr, tpr, label=label, color=color, linestyle=linestyle, linewidth=2.5)

# Top Left: HOBA
axes[0, 0].set_title('ROC curves for classifiers using the HOBA feature set', fontweight='bold')
for name, preds in hoba_predictions.items(): plot_roc(axes[0, 0], y_test, preds, f"{name} + HOBA", colors.get(name, 'orange'))

# Top Right: RFM
axes[0, 1].set_title('ROC curves for classifiers using the RFM feature set', fontweight='bold')
for name, preds in rfm_predictions.items(): plot_roc(axes[0, 1], y_test, preds, f"{name} + RFM", colors.get(name, 'orange'))

# Bottom Left: DL Models
axes[1, 0].set_title('ROC curves for deep learning classifiers using different feature sets', fontweight='bold')
for name in ['DBN', 'CNN', 'RNN']:
    plot_roc(axes[1, 0], y_test, hoba_predictions[name], f"{name} + HOBA", colors.get(name), '-')
    plot_roc(axes[1, 0], y_test, rfm_predictions[name], f"{name} + RFM", colors.get(name), '--')

# Bottom Right: ML Models
axes[1, 1].set_title('ROC curves for traditional machine learning classifiers using different feature sets', fontweight='bold')
for name in ['RF', 'SVM', 'BPNN']:
    plot_roc(axes[1, 1], y_test, hoba_predictions[name], f"{name} + HOBA", colors.get(name), '-')
    plot_roc(axes[1, 1], y_test, rfm_predictions[name], f"{name} + RFM", colors.get(name), '--')

for ax in axes.flat:
    ax.plot([0, 1], [0, 1], color='black', linestyle='--', linewidth=2)
    ax.set_xlim([0.0, 1.0]); ax.set_ylim([0.0, 1.02])
    ax.set_xlabel('False Positive Rate (FPR)', fontsize=12); ax.set_ylabel('True Positive Rate (TPR)', fontsize=12)
    ax.grid(True, linestyle='--', alpha=0.7)
    ax.legend(loc="lower right", fontsize=10, frameon=True, edgecolor='black')

```

```
plt.tight_layout()
plt.savefig('Final_ROC_Curves_Complete.png', dpi=300, bbox_inches='tight')
plt.show()
print("Success! The image is saved as 'Final_ROC_Curves_Complete.png'.")
```

--- STARTING RFM PIPELINE ---

Training BPNN (RFM)...

Training RF (RFM)...

Training SVM (RFM)...

Training CNN and RNN (RFM) on 4 features...

Training DBN (RFM)...

Extracting RFM Predictions...

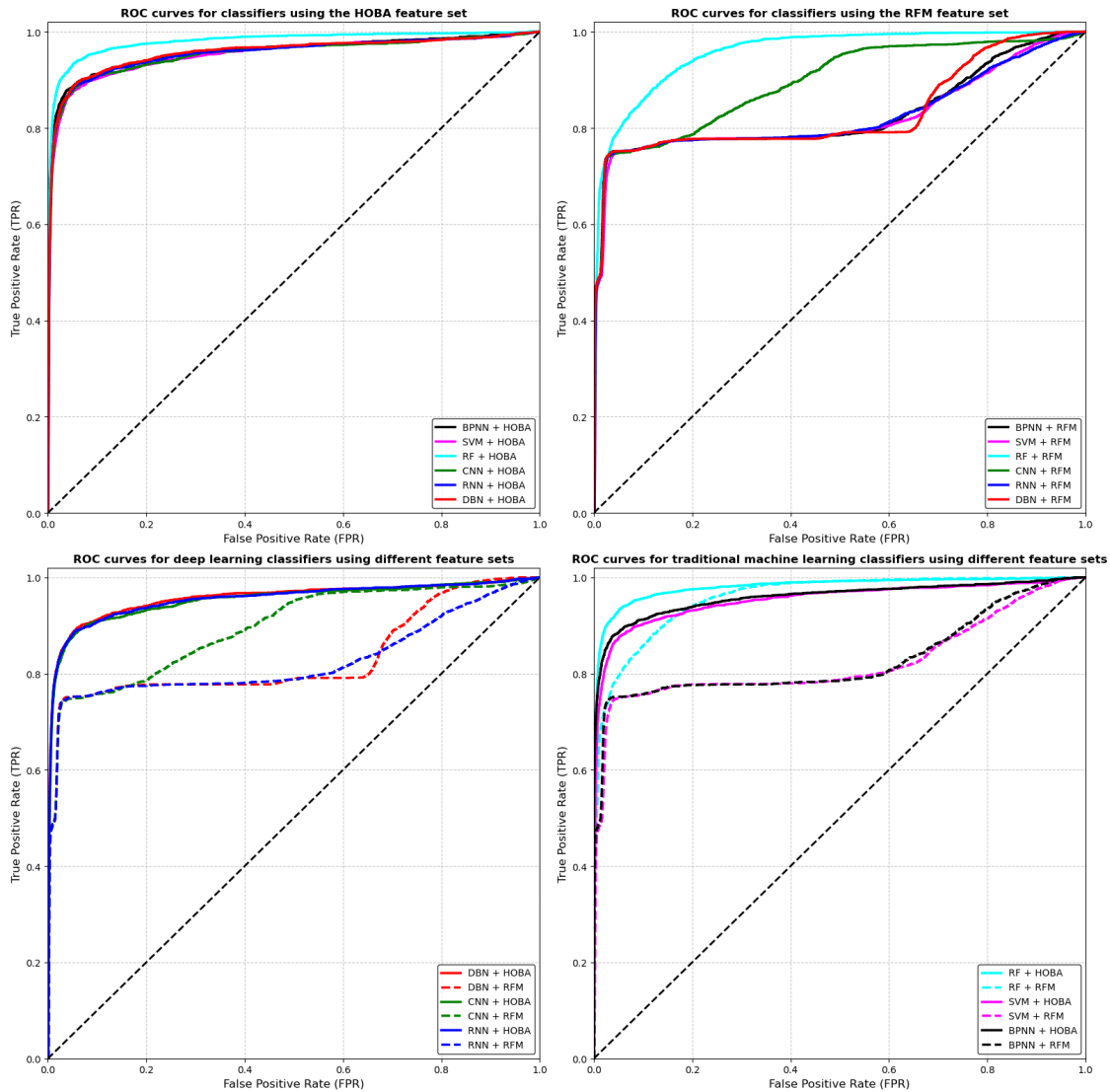
--- Table 5a: RFM Features (1% FPR) ---

	FPR	F1-Measure	Precision	Recall	Accuracy
Classifiers					
BPNN	1%	0.252	16.97%	48.86%	98.88%
SVM	1%	0.236	15.68%	47.93%	98.80%
RF	1%	0.300	19.65%	63.08%	98.86%
CNN	1%	0.240	15.94%	48.76%	98.81%
RNN	1%	0.241	16.00%	48.62%	98.82%
DBN	1%	0.242	16.04%	48.95%	98.81%

--- Table 5b: RFM Features (3% FPR) ---

	FPR	F1-Measure	Precision	Recall	Accuracy
Classifiers					
BPNN	3%	0.159	8.87%	74.41%	96.95%
SVM	3%	0.152	8.53%	72.07%	96.91%
RF	3%	0.157	8.78%	74.55%	96.91%
CNN	3%	0.158	8.82%	74.08%	96.94%
RNN	3%	0.158	8.87%	74.31%	96.95%
DBN	3%	0.159	8.91%	74.31%	96.97%

Drawing Final ROC Curves with DBN...



Success! The image is saved as 'Final_ROC_Curves_Complete.png'.

```
[31]: import pandas as pd
from sklearn.metrics import roc_auc_score

print("Calculating Area Under the ROC Curve (AUC) for all models...")

# Calculate AUC for both feature sets
auc_hoba = {name: roc_auc_score(y_test, preds) for name, preds in
    ↪hoba_predictions.items()}
auc_rfm = {name: roc_auc_score(y_test, preds) for name, preds in
    ↪rfm_predictions.items()}
```

```

# Combine into a single, clean DataFrame
df_auc = pd.DataFrame({
    'RFM Features (AUC)': auc_rfm,
    'HOBA Features (AUC)': auc_hoba
})

# Calculate the exact improvement delta
df_auc['Improvement with HOBA'] = df_auc['HOBA Features (AUC)'] - df_auc['RFM_Features (AUC)']

# Format everything to 4 decimal places for academic reporting
df_auc = df_auc.round(4)

print("\n--- Table: Area Under the ROC Curve (AUC) Comparison ---")
display(df_auc.sort_values(by='HOBA Features (AUC)', ascending=False))

```

Calculating Area Under the ROC Curve (AUC) for all models...

--- Table: Area Under the ROC Curve (AUC) Comparison ---

	RFM Features (AUC)	HOBA Features (AUC)	Improvement with HOBA
RF	0.9578	0.9805	0.0227
BPNN	0.8286	0.9590	0.1304
DBN	0.8320	0.9571	0.1251
RNN	0.8247	0.9544	0.1297
CNN	0.8942	0.9527	0.0585
SVM	0.8230	0.9522	0.1291

[]: